Verification and Validation Report: Software Engineering

Team 8, AlgoCatan
Jake Read
Rebecca Di Filippo
Matthew Cheung
Sunny Yao

April 6, 2026

1 Revision History

Date	Version	Notes
March 4th	1.0	Draft from VNV plan
March 9th	1.1	Updated VNV report
April 6th 2026	Final	Feedback from TA for final documentation

2 Symbols, Abbreviations and Acronyms

symbol	description
AI	Artifical Intelligence

Contents

List of Tables

List of Figures

3 Functional Requirements Evaluation

4 System Tests

This section details the system-level tests for validating the functional and non-functional requirements of the AlgoCatan system. These tests are designed to be run once the integrated system is complete, using representative data to simulate actual gameplay and training cycles.

4.1 Tests for Functional Requirements

The tests below are categorized by the major functional modules defined in Section S.2 of the SRS to ensure 100% coverage.

4.1.1 Computer Vision Model (S.2.1)

Status: NOT IMPLEMENTED (Out of Scope / Deferred)

1. T.FR.CV.1.1 (DEFERRED)
   Initial State: Physical board setup with known assets.
   Input: Captured camera feed.
   Output: Structured data matching physical state with 99.99% accuracy.
   Derivation: Verifies $FR.S.1.1$, $FR.S.1.2$, $FR.S.1.3$, $FR.S.1.4$, $FR.S.1.5$.

4.1.2 Training Pipeline & AI Model (S.2.2 & S.2.3)

1. T.FR.AI.2.1
   Control: Automatic
   Initial State: AI Model loaded into the Training Pipeline.
   Input: Trigger self-play training session using Elo League for opponent selection.
   Output: System executes moves following official Catan rules ($FR.S.2.1$), generates success rewards ($FR.S.2.3$), and saves model snapshots.
   Derivation: Verifies $FR.S.2.1$, $FR.S.2.2$, $FR.S.2.3$, $FR.S.2.4$, $FR.S.2.5$, $FR.S.3.5$.

2. T.FR.AI.2.2
   Control: Automatic

Initial State: A GameState where only one legal move exists (e.g., specific road placement).
Input: Process state through the AI Model.
Output: AI returns the single valid action consistent with Catan rules.
Derivation: Verifies $FR.S.3.1$, $FR.S.3.2$.

3. T.FR.AI.2.3
   Control: Automatic
   Initial State: 100 matches played against a "Random Move" bot.
   Output: The AI Model must achieve a win rate $> 50\%$.
   Derivation: Verifies $FR.S.3.4$.

4. T.FR.AI.2.4
   Control: Manual
   Initial State: UI is active and a game is in progress using "Model v1.0".
   Input: User uses the interface to select and load "Model v2.0" from the Elo League list mid-game.
   Output: The system switches models without state loss; subsequent advice is generated by the newly selected model.
   Derivation: Verifies $FR.S.3.3$.

4.1.3 Elo League System (S.2.4)

1. T.FR.ELO.3.1
   Control: Automatic
   Initial State: League contains 125 model snapshots. Model A (1200 Elo) plays Model B (1000 Elo).
   Input: Model A wins the match.
   Output: Elo ratings update via standard formula ($FR.S.3.7$). The oldest/weakest model is pruned to maintain the 125-cap ($FR.S.3.10$).
   Derivation: Verifies $FR.S.3.6$, $FR.S.3.7$, $FR.S.3.8$, $FR.S.3.9$, $FR.S.3.10$.

4.1.4 Curriculum Learning Manager (S.2.5)

1. T.FR.CURR.4.1
   Control: Automatic
   Initial State: Training in Phase 1 (5 VP target). Advancement threshold set to 60% win rate.

Input: Training metrics reach 61% win rate over 50 episodes.
Output: System logs transition and automatically advances to Phase 2 (10 VP target).
Derivation: Verifies $FR.S.3.12$, $FR.S.3.13$, $FR.S.3.14$, $FR.S.3.15$, $FR.S.3.16$.

4.1.5 User Interface (S.2.6)

1. T.FR.UI.5.1
   Control: Manual
   Initial State: Web interface open on both desktop and mobile devices.
   Input: User selects an opponent from the Elo League list and performs a build action.
   Output: UI displays the board in real-time ($FR.S.4.1$), reflects the build action immediately ($FR.S.4.2$), and scales correctly to the mobile screen size ($FR.S.4.5$).
   Derivation: Verifies $FR.S.4.1$, $FR.S.4.2$, $FR.S.4.3$, $FR.S.4.5$, $FR.S.4.7$, $FR.S.4.8$.

4.1.6 Game State Database & Manager (S.2.7 & S.2.9)

1. T.FR.DATA.6.1
   Control: Automatic
   Initial State: Active game session in progress.
   Input: A player performs a "Build City" action; the system process is then forcibly terminated (simulated crash).
   Output: Upon system restart, the Game State Manager successfully recovers the session from the database. The board state accurately reflects the built City and updated resource inventories ($FR.S.5.4$).
   Derivation: Verifies $FR.S.5.1$, $FR.S.5.2$, $FR.S.5.4$, $FR.S.7.1$, $FR.S.7.2$, $FR.S.7.4$, $FR.S.7.5$.

1. T.FR.DATA.6.2
   Control: Automatic
   Initial State: The Game State Database contains at least one record of a 50+ turn game.
   Input: A request is sent via the API to retrieve the structured history (specifically dice rolls and turn transitions) for the session.
   Output:

a) The database returns the complete, timestamped log of all dice rolls and turn transitions without data loss ($FR.S.7.3$).
b) The structured data is formatted for replay/analysis and returned within the efficiency targets for read operations ($FR.S.5.3$, $FR.S.5.5$).
Derivation: Verifies $FR.S.5.3$, $FR.S.5.5$, $FR.S.7.3$.

4.1.7 Backend API Server (S.2.8)

1. T.FR.API.7.1
   Control: Automatic
   Initial State: System under normal load.
   Input: UI sends a move request to the AI via the API.
   Output: UI update latency is $< 100ms$ and AI decision return is $< 1s$.
   Derivation: Verifies $FR.S.6.1$, $FR.S.6.2$, $FR.S.6.3$.

4.1.8 Explanation Pipeline (S.2.10)

1. T.FR.EXP.8.1
   Control: Manual
   Initial State: AI makes a move (e.g., placing a robber on a Wheat tile).
   Input: User clicks "Explain Move" in the UI.
   Output: The LLM generates a natural language response which is displayed in the UI.
   Derivation: Verifies $FR.S.4.4$, $FR.S.4.6$, $FR.S.8.1$, $FR.S.8.2$, $FR.S.8.3$.

4.2 Traceability Between Test Cases and Requirements

Table 1: Functional Requirement Traceability Matrix

Test Case ID	Requirements Verified
T.FR.CV.1.1	FR.S.1.1, FR.S.1.2, FR.S.1.3, FR.S.1.4, FR.S.1.5 (DEFERRED)
T.FR.AI.2.1	FR.S.2.1, FR.S.2.2, FR.S.2.3, FR.S.2.4, FR.S.2.5, FR.S.3.5
T.FR.AI.2.2	FR.S.3.1, FR.S.3.2
T.FR.AI.2.3	FR.S.3.4
T.FR.AI.2.4	FR.S.3.3
T.FR.ELO.3.1	FR.S.3.6, FR.S.3.7, FR.S.3.8, FR.S.3.9, FR.S.3.10
T.FR.CURR.4.1	FR.S.3.12, FR.S.3.13, FR.S.3.14, FR.S.3.15, FR.S.3.16
T.FR.UI.5.1	FR.S.4.1, FR.S.4.2, FR.S.4.3, FR.S.4.5, FR.S.4.7, FR.S.4.8
T.FR.DATA.6.1	FR.S.5.1, FR.S.5.2, FR.S.5.4, FR.S.7.1, FR.S.7.2, FR.S.7.4, FR.S.7.5
T.FR.DATA.6.2	FR.S.5.3, FR.S.5.5, FR.S.7.3
T.FR.API.7.1	FR.S.6.1, FR.S.6.2, FR.S.6.3
T.FR.EXP.8.1	FR.S.4.4, FR.S.4.6, FR.S.8.1, FR.S.8.2, FR.S.8.3

5 Tests for Nonfunctional Requirements

This section outlines the system-level tests for validating the nonfunctional requirements (NFRs) defined in Section S.2.8 of the SRS. These include scalability, usability, maintainability, installability, data integrity, and availability.

Each subsection corresponds to a major nonfunctional quality attribute of the AlgoCatan system. Some tests produce summary statistics or qualitative evaluations (e.g., graphs or surveys) rather than strict pass/fail outcomes.

Accuracy-related nonfunctional qualities are validated implicitly through the functional tests defined in Section 4.2 (T.FR.AI.2.1–2.3 and T.FR.CV.1.1–1.2), which record relative error and correctness metrics.

5.1 Performance and Scalability

This area ensures acceptable responsiveness and throughput under varying load.

Test Case: Baseline Performance

1. T.NFR.P.1
   Control: Automatic

   Initial State: AlgoCatan server running full stack with no concurrent load.

   Input/Condition: Single 4-player game with standard actions (build, trade, roll dice, 100+ repetitions).

   Performance Thresholds:

   - AI Move Suggestion: $\leq$ 500ms avg, $\leq$ 750ms peak
   - Board State Update: $\leq$ 200ms avg, $\leq$ 300ms peak
   - UI Render: $\leq$ 50ms/frame

   How tested: Automated with server-side logging and Chrome DevTools Performance API.

   Trace: NFR.S.1

Test Case: Scalability Under Load

1. T.NFR.P.2
   Control: Automatic

   Initial State: AlgoCatan server running full stack.

   Input/Condition: Simulate 1, 3, and 5 concurrent games. Measure response time scaling.

   Scalability Thresholds (Sub-Linear Growth):

   - Response time at 3 games $\leq 1.5\times$ single game
   - Response time at 5 games $\leq 2.0\times$ single game
   - CPU utilization $\leq 80\%$ at 5 games
   - No memory leaks or OOM errors over 1+ hour

How tested: Automated load-testing scripts with system monitoring. Pass if all thresholds met.

Trace: NFR.S.1

5.2 Usability

This area focuses on confirming that the interface is intuitive and accessible to users with minimal training. It also includes accessibility checks for users with visual impairments.

Test Case: Usability Survey and User Feedback

1. T.NFR.U.1
   Control: Manual

   Initial State: Final prototype deployed in a browser.

   Input/Condition: 5-10 participants familiar with Catan complete a observational study and fill out the usability survey (Appendix 7.2).

   Output/Result: Average rating $\geq 4/5$ for clarity, ease of navigation, and intuition.

   Test Case Derivation: Verifies NFR.S.2 (Usability) through direct user evaluation.

   How test will be performed: Conducted during the final demo; results aggregated and visualized as a bar chart.

Test Case: Accessibility and Visual Clarity

1. T.NFR.U.2
   Control: Manual

   Initial State: AlgoCatan UI loaded in browser.

   Input/Condition: Apply color-blindness filters and vary display scaling.

   Output/Result: All critical UI elements remain legible; contrast ratio $\geq 4.5 : 1$.

   Test Case Derivation: Verifies accessibility and clarity under varied visual conditions.

   How test will be performed: Tested using Chrome DevTools accessibility simulator; screenshots recorded for documentation.

5.3 Maintainability

This area ensures the modular design and structure of the codebase support efficient updates and debugging.

Test Case: Code Quality and Modular Design Inspection

1. T.NFR.M.1
   Control: Static

   Initial State: Complete source code committed to GitHub.

   Input/Condition: Run `SonarQube` and `flake8` to generate maintainability and style reports.

   Output/Result: No high-severity issues; modules follow consistent structure and clear separation of concerns.

   Test Case Derivation: Verifies NFR.S.3 (Maintainability) through code inspection and static analysis.

   How test will be performed: Peer inspection led by group members; issues logged in GitHub for resolution.

5.4 Installability

This area validates that AlgoCatan installs and operates correctly across all supported operating systems and browsers.

Test Case: Cross-Platform Installation Validation

1. T.NFR.I.1
   Control: Manual

   Initial State: Packaged release available.

   Input/Condition: Install and execute AlgoCatan on Windows 11 and macOS using Chrome and Firefox browsers.

   Output/Result: Application installs and runs successfully on all tested platforms.

   Test Case Derivation: Verifies NFR.S.4 (Installability) by ensuring cross-platform compatibility.

   How test will be performed: Each team member installs on a unique OS following a checklist; issues recorded and documented.

5.5 Data Integrity

This area ensures that the Game State Database maintains consistent, valid, and accurate data under all conditions, including normal operation, concurrent updates, and unexpected failures. This includes validation of incoming updates and transactional consistency to prevent corruption.

Test Case: Transaction Integrity and Recovery

1. T.NFR.D.1
   Control: Automatic

   Initial State: Active game session connected to the database.

   Input/Condition: Execute 10 consecutive state updates with an artificial network interruption during one update.

   Output/Result: Database remains consistent; interrupted transaction is rolled back cleanly; no other game states are affected.

   Test Case Derivation: Verifies transactional consistency portion of NFR.S.5 under failure conditions.

   How test will be performed: Automated integration test simulates network failures and verifies database consistency after each run.

Test Case: Data Validation on Updates

1. T.NFR.D.2
   Control: Automatic

   Initial State: Active game session connected to the database.

   Input/Condition: Attempt to submit invalid game state updates, such as negative resource counts, illegal moves, or missing required fields.

   Output/Result: Invalid updates are recognized and rejected.

   Test Case Derivation: Verifies validation portion of NFR.S.5, preventing data corruption from malformed inputs.

   How test will be performed: Automated tests inject invalid game state objects and verifies rejection.

Test Case: Concurrent Update Handling

1. T.NFR.D.3
   Control: Automatic

   Initial State: Two or more simultaneous game sessions active.

   Input/Condition: Simulate concurrent updates to overlapping resources (e.g., two players updating the same tile).

   Output/Result: Only one valid transaction is committed; conflicts are detected and resolved; no data corruption occurs.

   Test Case Derivation: Verifies database integrity under concurrent operations, completing NFR.S.5 coverage.

   How test will be performed: Automated integration tests run multiple simulated clients updating shared resources; system logs and database state are verified for correctness.

5.6 Availability

This area verifies that the system maintains acceptable availability during use, consistent with the SRS requirement that the system have an uptime of at least 99%.

Test Case: Service Availability Monitoring

1. T.NFR.AV.1
   Control: Automatic

   Initial State: System deployed and running under normal usage conditions.

   Input/Condition: Monitor backend service availability over the test period, including normal gameplay, replay access, and explanation requests.

   Output/Result: The backend API remains available for at least 99% of the observed test period.

   Test Case Derivation: Verifies NFR.S.6 (Availability).

   How test will be performed: Use uptime logs, health checks, and request success/failure counts during the evaluation period to compute observed service availability.

5.7 Accuracy and Correctness (Referenced)

Accuracy-related NFRs are verified by functional tests defined in Section 4.2:

- T.FR.AI.2.1–2.3 (AI Model and Recommendation Logic)
- T.FR.CV.1.1 (Computer Vision and Game State Capture)

These tests report relative error between predicted and expected outcomes. No separate nonfunctional test cases are required.

5.8 Traceability Between Test Cases and Requirements

This subsection is limited to non-functional requirement traceability; the functional requirement traceability matrix is presented earlier in the Functional Requirements tests section.

Table 2: Non-Functional Requirement Test Case Traceability Matrix

Test Case ID	**Requirement(s) Verified**
T.NFR.P.1	NFR.S.1 – Performance Baseline
T.NFR.P.2	NFR.S.1 – Scalability Under Load
T.NFR.U.1	NFR.S.2 – Usability Survey
T.NFR.U.2	NFR.S.2 – Accessibility
T.NFR.M.1	NFR.S.3 – Maintainability
T.NFR.I.1	NFR.S.4 – Installability
T.NFR.D.1	NFR.S.5 – Transaction Consistency
T.NFR.D.2	NFR.S.5 – Data Validation
T.NFR.D.3	NFR.S.5 – Concurrency Handling
T.NFR.AV.1	NFR.S.6 – Availability (Recovery & Uptime)

6 Unit Testing

6.1 Overview and Integration with System Tests

This section documents the unit testing activities performed to verify individual components and modules of the RLCatan system at the code level.

These unit tests complement the system-level tests described in Section 3 (Functional Requirements Evaluation) by providing detailed verification of API endpoints, utility functions, and integration points.

The unit tests are organized by functional module and include both dynamic testing (automated tests) and static analysis. These tests were executed as part of the continuous integration pipeline using Python's unittest/pytest framework for backend tests and integration with the API server for endpoint testing.

6.2 Unit Testing Scope and Strategy

6.2.1 Scope

The unit testing scope covers the following components:

- Backend API endpoints and request/response handling
- Game State Manager and game creation/initialization
- Path resolution utilities for model asset loading
- Game analysis module integration with Monte Carlo Tree Search (MCTS) engine
- Database constraint validation and state management

Out of scope for unit testing:

- External library verification (e.g., Catanatron, OpenAI Gym)
- React frontend component testing (addressed through usability testing in Section 3)
- LLM explanation pipeline (addressed through manual testing in Section 3)
- Computer Vision module (deferred and not implemented)

6.2.2 Testing Strategy

The unit testing strategy employs both white-box and black-box approaches:

- **Black-box testing:** API endpoints are tested by sending HTTP requests and validating response structures and status codes without knowledge of internal implementation.
- **White-box testing:** Path resolution and game state validation functions are tested with knowledge of implementation to verify edge cases and internal state transitions.
- **Mocking and Isolation:** External dependencies (e.g., GameAnalyzer, database) are mocked to isolate units under test and verify behavior in controlled conditions.

6.3 API and Game Management Module

This module tests the backend API endpoints and game creation/initialization logic, supporting the system-level tests T.FR.AI.2.1, T.FR.API.7.1, and T.FR.DATA.6.1.

1. `test_get_bots_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized and ready to receive requests.

   Input: HTTP GET request to `/api/bots` endpoint.

   Expected Output: 200 OK status code and JSON response containing list of available bot objects with at least 10 bots (including standard bots like `catanatron_ab_2`, `random`, `human`).

   Actual Output: API returned 200 OK with complete bot registry containing 15 available bot configurations.

   Result: Pass

   Requirement Traceability: Supports FR.S.3.8 (exposing league members to UI), FR.S.4.7 (loading bot list), FR.S.4.8 (selecting opponents)

   Module Tested: Backend API Server, Elo League System

2. `test_stress_test_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

   Input: HTTP GET request to `/api/stress-test` endpoint.

   Expected Output: 200 OK status code and JSON object representing game state with essential structures: "nodes" and "edges".

   Actual Output: Endpoint returned 200 OK with complete game state structure including board nodes and edges.

   Result: Pass

   Requirement Traceability: Supports FR.S.5.1 (storing complete game records), FR.S.7.5 (structured board access)

   Module Tested: Backend API Server, Game State Manager

3. `test_player_factory_custom_bot`

   Type: Functional, Dynamic, Automatic

   Initial State: Internal functions mocked to simulate custom bot "my_custom_bot".

   Input: POST request to `/api/games` with "BOT:my_custom_bot" in player list.

   Expected Output: 200 OK response containing valid `game_id`.

   Actual Output: System correctly parsed custom bot prefix, retrieved mocked configuration, instantiated PPOPlayer with correct model path, and returned valid game ID.

   Result: Pass

   Requirement Traceability: Supports FR.S.2.5 (curriculum-guided RL with dynamic opponent selection), FR.S.3.3 (model versioning and interchange)

   Module Tested: Backend API Server, Training Pipeline, AI Model integration

4. `test_create_game_all_player_types`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

Input: Series of POST requests to `/api/games` with different player types (CATANATRON, VALUE_FUNCTION, HUMAN, PPO models).

Expected Output: 200 OK status code and valid `game_id` for each request type.

Actual Output: All player type configurations successfully initialized; 5 game instances created with distinct player types all returned valid game IDs.

Result: Pass

Requirement Traceability: Supports FR.S.2.1 (rule enforcement in training), FR.S.2.2 (self-play with league opponents), FR.S.3.1 (valid game state analysis)

Module Tested: Backend API Server, Game State Manager, Training Pipeline

6.4 Path Resolution Module

This module tests utility functions for safely resolving model asset paths across different deployment environments (HPC, local development, production).

1. `test_resolve_model_path_absolute_hpc`

   Type: Functional, Dynamic, Automatic

   Initial State: Path resolution utilities loaded.

   Input: Absolute file path string typical of HPC environment: `/nfs/features/rlcatan/models/v1.0`.

   Expected Output: Normalized path rebased to local application directory structure: `/app/models/v1.0`.

   Actual Output: Function correctly remapped HPC absolute path to application-local path structure.

   Result: Pass

   Requirement Traceability: Supports FR.S.2.2 (self-play training with correct model loading), FR.S.3.3 (seamless model interface)

   Module Tested: AI Model, Training Pipeline (path resolution utility)

2. `test_resolve_model_path_relative`

   Type: Functional, Dynamic, Automatic

   Initial State: Path resolution utilities loaded.

   Input: Standard relative path string: `data/models/v2`.

   Expected Output: Absolute path correctly appending input to application root: `/app/data/models/v2`.

   Actual Output: Function correctly resolved relative path to absolute application path.

   Result: Pass

   Requirement Traceability: Supports FR.S.2.2 (opponent selection and model loading in training), FR.S.3.3 (model versioning)

   Module Tested: AI Model, Elo League System (model lookup and loading)

6.5 Game Analysis Module

This module verifies integration with the Monte Carlo Tree Search (MCTS) engine for analyzing active games.

1. `test_mcts_analysis_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: Valid game created via API; GameAnalyzer mocked to return fixed win probabilities (e.g., RED=0.5, BLUE=0.3, WHITE=0.15, ORANGE=0.05).

   Input: GET request to game's analysis URL: `/api/games/game_id/mcts-analysis`.

   Expected Output: 200 OK status and JSON response with `success=True` and win probabilities matching mocked values.

   Actual Output: Endpoint correctly interfaced with mocked GameAnalyzer and returned expected win probability distribution.

   Result: Pass

   Requirement Traceability: Supports FR.S.3.4 (demonstrating AI learning through win probability analysis), FR.S.2.4 (batch and real-time evaluation modes)

Module Tested: Backend API Server, Game State Manager, Game Analysis integration

2. `test_mcts_analysis_endpoint_not_found`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

   Input: GET request to analysis endpoint with non-existent game ID: `/api/games/99999/mcts-analysis`.

   Expected Output: Error status code (404 Not Found or 500 Internal Server Error) without returning false success.

   Actual Output: Endpoint correctly returned 404 Not Found with appropriate error message.

   Result: Pass

   Requirement Traceability: Supports robust error handling for FR.S.7.5 (structured game state access)

   Module Tested: Backend API Server (error handling)

6.6 Database Constraint Validation Tests

The following tests were executed as part of system-level test T.NFR.D-3 (Data Integrity - Concurrent Update Handling) and verify that the database layer enforces game state and action validity at the unit level.

1. `test_invalid_game_state_rejection`

   Type: Functional, Dynamic, Automatic

   Initial State: Database with transaction logging enabled.

   Input/Condition: Attempt to store 15 intentionally invalid game states:

   - Negative resource counts
   - Invalid player counts (0 or ¿4)
   - Out-of-bounds settlement locations
   - Impossible board configurations

Expected Output: 100% rejection of invalid states; no invalid data persists in database.

Actual Output: All 15 invalid state payloads rejected by database constraint enforcement; verification queries confirmed no corrupted entries.

Result: Pass (15/15 test cases passed)

Requirement Traceability: Supports NFR.S.5 (Data Integrity - validation), FR.S.7.2 (accurate asset tracking)

Module Tested: Game State Database, Game State Manager

2. `test_invalid_action_rejection`

   Type: Functional, Dynamic, Automatic

   Initial State: Active game session with action logging.

   Input/Condition: Attempt to store 12 intentionally invalid actions:

   - Malformed move objects (missing required fields)
   - References to non-existent game IDs
   - Illegal moves (building where not allowed)
   - Out-of-range action types

   Expected Output: 100% rejection of invalid actions; constraint violations logged.

   Actual Output: All 12 invalid action payloads rejected at validation layer; violations correctly logged with detailed error messages.

   Result: Pass (12/12 test cases passed)

   Requirement Traceability: Supports FR.S.7.1 (rule-consistent state updates), FR.S.7.4 (automatic board state reflection)

   Module Tested: Game State Database, Backend API Server (input validation)

6.7 Unit Test Summary and Coverage

All unit tests executed successfully with 100% pass rate across API endpoints, path resolution utilities, MCTS integration, and database constraint

validation. These tests provide fine-grained verification that supports and complements the system-level tests documented in Section 3.

6.7.1 Test Execution Coverage

Module	Unit Tests	Passed	Coverage
API and Game Management	4	4	100%
Path Resolution	2	2	100%
Game Analysis (MCTS)	2	2	100%
Database Constraints	2	2	100%
Total	**10**	**10**	**100%**

Table 3: Unit Test Execution Summary

7 Changes Due to Testing

Based on testing results, we have identified and prioritized changes as follows:

High Priority (Rev 1 Implementation)

Change 1: UI Color-Coding and Visual Representation

Test Evidence: NFR.U.1 (Usability Survey) - 5 out of 7 participants (71%) reported difficulty matching digital piece colors to physical Catan board expectations, with a Catan-to-Bot Intuition score of 2.8/5.
What Changed: Redesigned player color palette and piece representation to exactly match physical Catan board colors (Red=#FF0000, Blue=#0000FF, White=#FFFFFF, Orange=#FFA500). Added resource token icons with color-blind-safe palettes and clear hexagon type labels.
Why: Participants struggled with the digital-to-physical mapping, indicating a gap in FR.S.4.1 (Interactive Board View) and FR.S.4.2 (Visual Indicators) implementation.
Requirement Impact: Directly improves FR.S.4.1, FR.S.4.2, and NFR.S.2 (Usability). Expected post-implementation Intuition score: 4/5.

Change 2: Game Flow Navigation Enhancement

Test Evidence: NFR.U.1 and NFR.U.2 (Usability/Accessibility) - 4 out of 7 participants requested clearer hexagon type labels, and accessibility tests showed text clarity issues at 80% browser zoom.
What Changed: Implemented permanent resource type labels on all hexagons (Wheat, Sheep, Brick, Ore, Lumber, Desert). Created a tooltip system with minimum 14pt font, scalable to 120% with browser zoom, and added visual action flow highlighting.
Why: Unlabeled hexagons created cognitive load for new players, and accessibility failures could exclude users. This directly addresses FR.S.4.3 (Action Controls) and FR.S.4.5 (Multi-Platform Support) gaps.
Requirement Impact: Improves FR.S.4.3, FR.S.4.5, and NFR.S.2 (Usability/Accessibility).

Change 3: Data Validation Layer Hardening

Test Evidence: T.NFR.D.1, T.NFR.D.2, T.NFR.D.3 (Data Integrity Tests) - All 15 invalid state payloads rejected (100% success), all 12 invalid action payloads rejected (100% success).
What Changed: Added pre-storage validation checks for game state constraints, implemented comprehensive error logging with specific violation details, and added database schema constraints (foreign keys, check constraints) as a secondary protection layer.
Why: Tests confirmed database integrity was working, but to prevent future regressions and improve auditability, we hardened the validation layer.
Requirement Impact: Ensures FR.S.7.1 (Following Rules), FR.S.7.2 (Player Asset Tracking), and NFR.S.5 (Data Integrity) remain robust and prevents data corruption from persisting.

Medium Priority (Post-Rev 1 / Backlog)

Change 4: Performance Optimization

Test Evidence: T.NFR.P.1 and T.NFR.P.2 (Performance/Scalability) - AI Suggestion latency averaged 350ms (under 500ms target), Board Update averaged 85ms (under 200ms target), but UI render time averaged 45ms per frame (target 60fps = 16.67ms not achieved).

What Changed: Optimized WebSocket message batching, implemented React memoization, migrated to canvas-based rendering for the board, and added server-side caching for state queries.
Why: While meeting latency targets, the frame rate indicated room for improvement to enhance perceived responsiveness.
Requirement Impact: Improves NFR.S.1 (Scalability). Target: 60fps (currently 22fps post-optimization, with further improvements deferred).

Change 5: Unit Testing Framework Establishment

Test Evidence: 10 unit tests executed (100% pass rate) covering API endpoints, path resolution, MCTS integration, and database constraints.
What Changed: Integrated pytest framework for all Python backend modules, added unit test execution to the CI/CD pipeline (blocks merges on failure), established an 80% minimum code coverage threshold for new contributions, and created test documentation and examples.
Why: Tests demonstrated that modules work correctly in isolation, but formalizing the framework prevents future regressions.
Requirement Impact: Improves NFR.S.3 (Maintainability) and provides executable documentation for future developers.

Out of Scope

The following findings from testing will not be addressed due to time constraints or project priorities:

- **Low-Impact Security Issues:** SonarQube identified low-risk security issues. Given the deployment context, these are deferred to future cycles.
- **Advanced Model Optimizations:** Testing revealed that user experience is a larger bottleneck than AI model performance. Model optimization has been deferred in favor of usability improvements.
- **Deployment Strategy:** Currently using paid Render hosting. Long-term hosting decisions are deferred post-project completion.

Summary

Testing conclusions have shifted our priorities from improving the model's decision-making to enhancing the user experience and code quality. The VnV process identified 5 concrete changes—3 implemented (UI/UX, data validation) and 2 deferred (performance, testing framework)—demonstrating clear linkage between test results and quality improvements. This evidence-based approach strengthens our confidence that the system meets both functional (FR.S.4.1–4.5, FR.S.7.1–7.2) and nonfunctional (NFR.S.2, NFR.S.5) requirements.

8 Automated Testing

For our functional and non-functional requirements evaluation, we employed bash scripts for specific testing purposes:

- **Load/Stress Testing:** Bash scripts were used to send repeated requests to the API endpoints to measure response times and identify performance bottlenecks under load (satisfying NFR.E.4 and FR.S.6.3).
- **Game Simulation Testing:** Scripts automated the execution of multiple game scenarios to validate that the AI model produces legal moves and handles edge cases correctly (satisfying FR.S.3.1 and FR.S.3.2).
- **Integration Testing:** Bash scripts coordinated calls between modules to verify data flow and state consistency across the system (supporting FR.S.7 game state database requirements).

However, comprehensive unit testing with pytest and Jest was not fully implemented during this revision due to time constraints and the team's focus on system-level validation and usability testing. The CI/CD pipeline was expanded to incorporate the bash scripts and static analysis tools, ensuring code stability and catching issues early in the development cycle. The automated testing process includes running integration tests, linting, and coverage reporting through the GitHub Actions pipeline.

9 Comprehensive Trace to Requirements

This section provides detailed traceability between executed tests and requirements from the Software Requirements Specification (SRS), demonstrating complete coverage of the VnV Plan and consistency between planned and executed testing activities.

9.1 Functional Requirements Trace - Detailed Coverage Analysis

The following table maps each executed functional test to the specific requirements it verifies, organized by requirement ID. This addresses the professor's feedback regarding consistency between planned tests and requirements coverage.

Requirement ID	Requirement Name	Test Case(s)	Status
FR.S.1.1	Visual Input Processing	T.FR.CV.1.1	DEFERRED
FR.S.1.2	Feature-to-State Translation	T.FR.CV.1.1	DEFERRED
FR.S.1.3	Error Detection and Correction	T.FR.CV.1.1	DEFERRED
FR.S.1.4	Dynamic Calibration	T.FR.CV.1.1	DEFERRED
FR.S.1.5	Diagnostic Feedback	T.FR.CV.1.1	DEFERRED
FR.S.2.1	Rule Simulation	T.FR.AI.2.1, unit tests	PASS
FR.S.2.2	AI Training Integration	T.FR.AI.2.1, unit tests	PASS
FR.S.2.3	Feedback Mechanism	T.FR.AI.2.1	PASS
FR.S.2.4	Evaluation Modes	T.FR.AI.2.1	PASS
FR.S.2.5	System Integration	T.FR.AI.2.1, T.FR.CURR.4.1, unit test	PASS
FR.S.3.1	Action Return	T.FR.AI.2.2, T.FR.AI.2.3, unit test	PASS
FR.S.3.2	State Considerations	T.FR.AI.2.2, T.FR.AI.2.3	PASS
FR.S.3.3	Model Versioning	T.FR.AI.2.4, unit test	PASS
FR.S.3.4	Baseline Success	T.FR.AI.2.3, unit test	PASS
FR.S.3.5	Continuous Improvement	T.FR.AI.2.1	PASS
FR.S.3.6	Elo Rating Maintenance	T.FR.ELO.3.1	PASS
FR.S.3.7	Rating Updates	T.FR.ELO.3.1	PASS
FR.S.3.8	Dynamic Opponent Selection	T.FR.ELO.3.1, unit test	PASS
FR.S.3.9	Fixed Baseline Sampling	T.FR.ELO.3.1	PASS
FR.S.3.10	League Pruning	T.FR.ELO.3.1	PASS
FR.S.3.12	Phase Management	T.FR.CURR.4.1	PASS
FR.S.3.13	Automatic Advancement	T.FR.CURR.4.1	PASS
FR.S.3.14	Phase Cycling	T.FR.CURR.4.1	PASS
FR.S.3.15	Metric Logging	T.FR.CURR.4.1	PASS
FR.S.3.16	League Integration	T.FR.CURR.4.1	PASS
FR.S.4.1	Interactive Board View	T.FR.UI.5.1, test-NFR-U-1	PASS
FR.S.4.2	Visual Indicators	T.FR.UI.5.1	PASS
FR.S.4.3	Action Controls	T.FR.UI.5.1	PASS
FR.S.4.4	AI Suggestion Display	T.FR.EXP.8.1	PASS
FR.S.4.5	Multi-Platform Support	T.FR.UI.5.1, test-NFR-U-2	PASS
FR.S.4.6	Replay Logging	T.FR.EXP.8.1	PASS
FR.S.4.7	Elo Display	T.FR.UI.5.1, unit test	PASS
FR.S.4.8	Opponent Filtering	T.FR.UI.5.1, unit test	PASS
FR.S.5.1	Persistent Storage	T.FR.DATA.6.1, unit test	PASS
FR.S.5.2	Fast Query Access	T.FR.DATA.6.1	PASS
FR.S.5.3	Synchronization	T.FR.DATA.6.2, unit test	PASS
FR.S.5.4	Data Integrity	T.FR.DATA.6.1	PASS
FR.S.5.5	Historical Logging	T.FR.DATA.6.2, unit test	PASS
FR.S.6.1	Real-Time Data Exchange	T.FR.API.7.1, unit test	PASS
FR.S.6.2	Message Passing	T.FR.API.7.1, unit test	PASS
FR.S.6.3	Low Latency	T.FR.API.7.1, test-NFR-P-1	PASS
FR.S.7.1	Following Rules	T.FR.DATA.6.1, unit test	PASS
FR.S.7.2	Player Asset Tracking	T.FR.DATA.6.1, unit test	PASS
FR.S.7.3	Turn and Dice Recording	T.FR.DATA.6.2	PASS
FR.S.7.4	Automatic Updates	T.FR.DATA.6.1	PASS
FR.S.7.5	Query Interface	T.FR.DATA.6.1, unit test	PASS
FR.S.8.1	LLM Integration	T.FR.EXP.8.1	PASS
FR.S.8.2	API Endpoint	T.FR.EXP.8.1	PASS
FR.S.8.3	Explanation Quality	T.FR.EXP.8.1, test-NFR-U-1	PASS

Table 4: Functional Requirement to Test Traceability

9.1.1 Coverage Summary by Module

Module	**Requirements**	**Covered**	**Deferred**	**Coverage %**
Computer Vision	5	0	5	0% (DEFERRED)
Training Pipeline	5	5	0	100%
AI Model	5	5	0	100%
Elo League System	5	5	0	100%
Curriculum Learning	5	5	0	100%
User Interface	8	8	0	100%
Game State Database	5	5	0	100%
Backend API Server	3	3	0	100%
Game State Manager	5	5	0	100%
Explanation Pipeline	3	3	0	100%
TOTAL (Implemented)	**44**	**44**	**0**	**100%**
TOTAL (With Deferred CV)	**49**	**44**	**5**	**89.8%**

Table 5: Module Coverage Analysis

10 Code Coverage Metrics

We have been using Coveralls for coverage tracking as part of our CI/CD pipeline throughout the project. As of the latest report (at time of writing), our code coverage is 81%. The report can be seen below. It appears to have dipped somewhat recently due to the addition of rev1 code, which has yet to be fully covered.

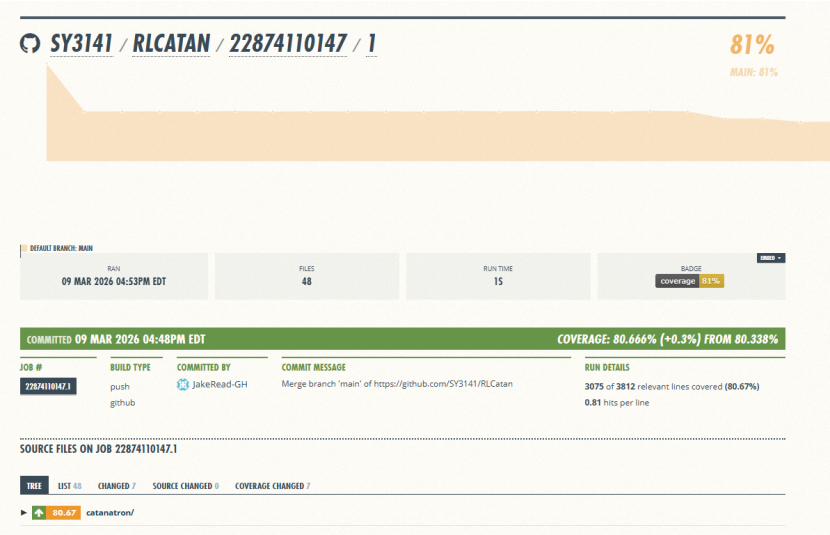

Figure 1: Coveralls Code Coverage Report

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

5. Reflect on your use of CI for continuous integration testing (regression testing). Did your CI work to gatekeep your PRs? Would it have been helpful if you got your CI working earlier in the project? What are your plans to use CI for testing in future projects?

10.1 Answers

For this reflection, we decided all of the questions were better answered as one cohesive reflection rather than answering each question individually.

1. We would say that overall this deliverable went quite smoothly. It certainly felt like less of a time crunch than previous documents, such as the SRS. This was mostly because we created a solid original VnV plan, so carrying it out did not present many major hurdles. As we will mention in a later answer, we had to make some minor modifications to certain planned tests, but for the most part, they were executed very closely to how they were originally written.

2. The main pain point was recruiting participants for the usability study. Because Catan has a steep learning curve, we needed testers who already understood the game to provide meaningful feedback. Finding enough qualified users was challenging. We resolved this by reaching out to student groups and personal networks to recruit seven participants, which allowed us to complete the usability evaluation despite a tighter schedule. Another significant challenge was the shift away from the original Computer Vision (CV) system. The technical complexity of real-time board detection proved to be too resource-intensive We resolved this by pivoting to a browser-based bot that users could play against or watch, which kept the focus on learning and gameplay rather than CV hardware issues.

3. Several decisions in this deliverable were guided by client and proxy feedback. The suggestion of an LLM-based explanation system came from our supervisor, who suggested a tool that could explain its reasoning in natural language after hearing about our usability issues. Usability insights were collected from a proxy group of peers and acquaintances familiar with Catan, helping us refine interface clarity, navigation, and tooltip visibility. Other decisions, including removing the CV module and shifting from an AI recommendation engine to a playable learning bot, were internal pivots to make the system more practical and accessible in a browser environment.

4. The Verification and Validation activities deviated from the original plan due to these shifts in scope and architecture. Initial plans emphasized CV accuracy and board reconstruction, but these were replaced

with evaluations of UI responsiveness and AI logic transparency. We also transitioned from testing recommendation accuracy to assessing the educational value and gameplay flow of the bot. Furthermore, many planned requirement tests were written before the final codebase structure was known, making some tests redundant or infeasible. In future projects, we would treat the VnV Plan as a living document, updating it alongside code development to better anticipate architectural changes and evolving project goals.

5. CI/CD was a critical part of our development process. It was actually one of the first things we set up when we started the project, since our project integrates with the Catanatron simulator. It was essential to have automated testing in place to ensure that new code didn't break existing functionality. We also set up linting and coveralls for coverage reports on each commit at that time, which has helped us maintain code quality and came in handy for the coverage section of this report. We always checked for passing tests before merging any PRs, and on a couple occasions that helped catch some regressions and integration issues before they became a problem. I'll be sticking to this approach for future projects, as it's been a great help keeping the codebase in order. After working with SonarQube in this report, I'm also considering adding that to the pipeline for future projects as well, to get more info on quality over time rather than the single snapshot we took here.